

Maquetación: cómo hacer inclusivos nuestros sitios web

10 ejercicios prácticos — SASS, responsividad, accesibilidad y frameworks

DOCUMENTO DE REFERENCIA

Frontend 03

Estos ejercicios repasan los preprocesadores CSS (SASS), la flexibilidad y adaptabilidad responsive, la accesibilidad web y los frameworks CSS más usados.

Fácil (1-3)

Fácil-Medio (4-7)

Medio (8-10)

1 Qué es un preprocesador CSS

FÁCIL

Explica qué es un preprocesador CSS y para qué sirve. Nombra al menos dos preprocesadores distintos a SASS mencionados en el documento.

2 Variables en SASS

FÁCIL

Declara una variable SASS `$color-primario` con el valor `#2331d4` y úsala dentro de una regla `.boton { color: ... }.`

3 Variables CSS nativas vs SASS

FÁCIL

Explica una diferencia clave entre las variables CSS nativas (`--mi-variable`) y las variables de SASS (`$mi-variable`) en cuanto a cuándo se resuelven (compilación vs en el navegador).

4 Nesting en SASS

FÁCIL-MEDIO

Reescribe usando nesting de SASS el siguiente CSS, que repite el selector padre:

```
#main-menu { color: black; }
#main-menu h2 { font-size: 20px; }
#main-menu a { text-decoration: none; }
```

5 Crear un mixin

FÁCIL-MEDIO

Crea un mixin SASS llamado `flex-center` que aplique `display: flex; justify-content: center; align-items: center;`, y utilízalo con `@include` dentro de una clase `.contenedor`.

6 Breakpoints y media queries

MEDIO

Enumera los cinco breakpoints típicos mencionados en el documento (sm, md, lg, xl, xxl) con su valor en píxeles, y escribe una media query en CSS puro que aplique `flex-direction: column;` a partir de 576px de ancho de pantalla.

7 Mobile first

MEDIO

Explica en qué consiste la metodología "mobile first" y por qué el documento recomienda empezar a diseñar por el tamaño de pantalla del móvil antes que por el de escritorio.

8 Buenas prácticas de accesibilidad

MEDIO

Enumera cuatro buenas prácticas de accesibilidad mencionadas en el documento (por ejemplo, sobre imágenes, contraste o formularios), e indica qué nivel de la WCAG (A, AA o AAA) se suele exigir como mínimo en proyectos para clientes públicos.

9 Bootstrap vs Tailwind

MEDIO

Compara Bootstrap y Tailwind según lo explicado en el documento: ¿cómo aplica cada uno los estilos y qué tipo de proyecto encajaría mejor con cada enfoque?

10 Ejercicio integrador

MEDIO

Diseña en SASS un botón responsive y accesible: define una variable de color, un mixin `flex-center` para centrar su contenido, una media query para reducir su tamaño en pantallas pequeñas (menos de 576px) y añade un comentario indicando qué atributo HTML usarías para garantizar un buen contraste y accesibilidad del texto del botón.

Soluciones orientativas

1. Un preprocesador CSS permite generar hojas de estilo a partir de una sintaxis extendida (variables, anidación, funciones...) que después se compila a CSS estándar. Otros preprocesadores: LESS, Stylus, PostCSS.
2. `$color-primario: #2331d4; .boton { color: $color-primario; }`
3. Las variables CSS nativas se resuelven en el navegador y pueden cambiar dinámicamente (incluso con JavaScript); las variables SASS se compilan antes de llegar al CSS final, quedando fijadas con el valor que tuvieran en tiempo de compilación.
4. `#main-menu { color: black; h2 { font-size: 20px; } a { text-decoration: none; } }`
5. `@mixin flex-center { display: flex; justify-content: center; align-items: center; } .contenedor { @include flex-center; }`
6. sm: 576px, md: 768px, lg: 993px, xl: 1200px, xxl: 1400px. `@media screen and (min-width: 576px) { .elemento { flex-direction: column; } }`
7. Mobile first propone diseñar y construir primero los estilos para la pantalla de un móvil, y después ir añadiendo breakpoints y media queries para pantallas mayores, porque el móvil es el dispositivo más usado y limita más los recursos y el espacio disponible.
8. Por ejemplo: vigilar el contraste de colores; usar siempre `alt` descriptivo en imágenes; añadir `captions` a los vídeos; establecer correctamente el atributo `lang` del `<html>`. El nivel mínimo habitual exigido es AA (doble A).
9. Bootstrap ofrece componentes y clases predefinidas muy completas y personalizables, ideal para montar interfaces rápido con una estética coherente; Tailwind expone utilidades de bajo nivel aplicadas directamente en el HTML, dando más control y flexibilidad de diseño a cambio de escribir más clases.
10. El botón usaría `$color-primario` para el fondo, `@include flex-center` para centrar su contenido, una media query `@media (max-width: 576px)` para reducir el `padding` / `font-size`, y el texto debería tener suficiente contraste respecto al fondo (comprobable con herramientas como Axe o Lighthouse) para cumplir accesibilidad.

Practica primero sin mirar las soluciones — es la mejor forma de consolidar lo aprendido.